DarkSage

Technical Architecture

Version 0.4.2 — Draft

Owner: TheSinnerMan

Classification: Internal

Wisdom Over Noise

Source repository: LightSwornPVP/DarkSage
Source baseline commit: 8b5cf5bd0341d5d6f70c67302c8bce2069d0929a
Generation date: 2026-07-25T16:51:31-04:00
Document ID: DS-004

Document Control

Document ID	DS-004
Title	Technical Architecture
Version	0.4.2
Status	Draft
Owner	TheSinnerMan
Classification	Internal
Repository	LightSwornPVP/DarkSage
Created	2026-07-23
Last Updated	2026-07-25

Status lifecycle: Draft → Under Review → Approved → Superseded/Deprecated.

Generated from DS-004 v0.4.2 — baseline 8b5cf5bd0341d5d6f70c67302c8bce2069d0929a

Revision History

Version	Date	Author	Summary
0.4.2	2026-07-25	TheSinnerMan / Keeper	Independent-audit blocker-repair (Blockers 2, 3): extended DS-ARC-027 with the DS-JRN-007 retention/deletion architecture; extended DS-ARC-028 with a cross-reference to DS-SCA-029's Trade Intelligence Package security authority (DS-008). No prior section renumbered or restated.
0.4.1	2026-07-25	TheSinnerMan / Keeper	Independent-audit repair (H1, H3): added §§16b–16d (`DS-ARC-026` Research Intelligence Architecture, `DS-ARC-027` Journal & Review Architecture, `DS-ARC-028` Canonical Trade Intelligence Package Service) closing the H1 architecture-traceability gap for Research Intelligence, Journal & Review Intelligence, and the Trade Intelligence Package. Renumbered the Founder Vision Completion section from the misnumbered "## 24." (skipping §§22–23) to the correct next-available "## 22." No prior section's content changed.
0.4.0	2026-07-25	TheSinnerMan / Keeper	Founder Vision Completion amendment and cross-volume traceability, including Discord notification boundaries where applicable.
0.1.0	2026-07-23	TheSinnerMan	Document control scaffold and Foundational Direction statement created; no detailed normative content.
0.2.0	2026-07-24	TheSinnerMan	First substantial controlled draft. Formalizes the repository's pre-existing, authoritative `ARCHITECTURE.md` (31 sections) into Codex-governed, testable `DS-ARC-NNN` requirements, cross-referenced to DS-001, DS-002, DS-003, and ADR-001–004. `ARCHITECTURE.md` remains the detailed implementation reference (full indicator/pattern/metric catalogs, exact module lists); DS-004 states the governance-critical architectural contracts as traceable, auditable requirements.

Generated from DS-004 v0.4.2 — baseline 8b5cf5bd0341d5d6f70c67302c8bce2069d0929a

Version	Date	Author	Summary
0.3.0	2026-07-24	TheSinnerMan	Repair pass addressing independent audit findings DS-004-A01 through A03. **A01:** reclassified `DS-ARC-011/012` (TradeValidationPipeline/Risk-Permissions integration, Phase 7) and `DS-ARC-013` (AI Provider Interface, Phase 6) from Committed/MVP to Planned; narrowed `DS-ARC-004` (Backend Service Architecture) and `DS-ARC-005` (Shared Models) to their genuine Phase-1 module/model subset, with the remainder explicitly Planned. Added a document-level Release Classification note distinguishing "implementation phase" from "governance boundary." **A02:** added six new requirements (`DS-ARC-020` through `DS-ARC-025`) covering Strategy Performance/DNA, Broker/Auto-Trader architecture, Emergency Stop/Flatten, Portfolio architecture, Mobile control security, and the complete Trading Knowledge Engine architecture (`ARCHITECTURE.md` §31) — all previously omitted. **A03:** gave `DS-ARC-003` (Mobile Client) concrete, testable acceptance criteria now (client-only, backend-authoritative boundaries), while its implementation timing remains Phase 9/Planned.

Table of Contents

Document Control 2
Revision History 3
Table of Contents 5
Foundational Direction 7
1. Purpose 7
2. Scope 7
3. Audience 7
4. Definitions 8
5. Client/Server Topology 8
DS-ARC-001 — Client/Server Separation . . . 8
DS-ARC-002 — Desktop Client . . . 9
DS-ARC-003 — Mobile Client . . . 9
6. Backend Services 10
DS-ARC-004 — Backend Service Architecture . . . 10
DS-ARC-005 — Shared Models . . . 11
7. Market Data Architecture 11
DS-ARC-006 — Market Data Provider Abstraction . . . 11
8. Indicator Engine 12
DS-ARC-007 — Single Shared Indicator Engine . . . 12
9. Chart Architecture 12
DS-ARC-008 — Chart Adapter Abstraction . . . 12
10. Strategy and Backtesting Architecture 13
DS-ARC-009 — Strategy Interface and Versioning . . . 13
DS-ARC-010 — Backtest Engine Architecture . . . 13
11. Canonical Trade Validation Pipeline 14
DS-ARC-011 — TradeValidationPipeline Enforcement . . . 14
12. Risk and Permissions Architecture 15
DS-ARC-012 — Risk Engine and Permissions Engine as Distinct Pipeline Stages . . . 15
13. AI Architecture 16
DS-ARC-013 — AI Provider Interface . . . 16
DS-ARC-014 — Local-First AI . . . 16
DS-ARC-015 — AI Provider Credential Handling . . . 17

Generated from DS-004 v0.4.2 — baseline 8b5cf5bd0341d5d6f70c67302c8bce2069d0929a

14. Database and Caching 17
DS-ARC-016 — Database Strategy . . . 17
DS-ARC-017 — Caching Strategy . . . 18
15. Deployment Stages 18
DS-ARC-018 — Staged Deployment Model . . . 18
16. Core Architectural Rules 19
DS-ARC-019 — No Tight Coupling . . . 19
16a. Missing Architecture Contracts (added in the DS-004-A02 repair) 19
DS-ARC-020 — Strategy Performance and Strategy DNA Architecture . . . 19
DS-ARC-021 — Broker and Auto-Trader Architecture . . . 20
DS-ARC-022 — Emergency Stop and Emergency Flatten Architecture . . . 20
DS-ARC-023 — Portfolio Architecture . . . 21
DS-ARC-024 — Mobile Control Security Architecture . . . 21
DS-ARC-025 — Trading Knowledge Engine Architecture . . . 21
16b. Research Intelligence Architecture (added in the independent-audit H1 repair) 22
DS-ARC-026 — Research Intelligence Architecture . . . 22
16c. Journal & Review Architecture (added in the independent-audit H1 repair) 23
DS-ARC-027 — Journal & Review Architecture . . . 23
16d. Canonical Trade Intelligence Package Service (added in the independent-audit H1 repair) 24
DS-ARC-028 — Canonical Trade Intelligence Package Service . . . 24
17. Non-Goals 25
18. Dependencies 25
19. Risks and Constraints 26
20. Verification Approach 26
21. References 26
Appendix A — Open Questions 26
22. Founder Vision Completion Architecture 27

Foundational Direction

Desktop-first; local-first where practical; deterministic financial calculations; presentation independence; user-controlled automation; explainability by default.

1. Purpose

DS-004 is the Codex-governed technical architecture specification for DarkSage. It does not invent architecture: the repository already contains a mature, pre-existing, authoritative engineering specification (`ARCHITECTURE.md`, `PROJECT_SPEC.md`, `ROADMAP.md`) that predates this Codex volume. DS-004's role is to translate the governance-critical parts of that specification — the parts that enforce ADR-001–004's boundaries and DS-002/DS-003's requirements — into testable, traceable, ID'd requirements, and to point to `ARCHITECTURE.md` as the source of truth for exhaustive implementation detail (full indicator lists, full pattern catalogs, full module trees) that does not need duplication here.

2. Scope

This document governs the architectural contracts that enforce:

- the client/server topology and the rule that clients never contain critical trading logic;
- provider/engine abstraction boundaries (market data, charts, AI);
- the canonical `TradeValidationPipeline` and Sage's inability to bypass it (ADR-002);
- the deterministic-calculation/AI-advisory boundary (ADR-003);
- database, caching, and deployment-stage sequencing; and
- the core architectural rules against tight coupling (`ARCHITECTURE.md` §30).

DS-004 does not restate `ARCHITECTURE.md`'s full detail (e.g., the complete indicator list, the complete chart-pattern list, every backend module) — those remain authoritative in `ARCHITECTURE.md` itself and are referenced, not copied. DS-004 does not govern database schema detail (DS-005), API contracts (DS-006), UI/UX system detail (DS-007), or security architecture detail (DS-008); where a DS-004 requirement borders one of those, it states the architectural contract and delegates implementation detail forward.

Release Classification note (added in the DS-004-A01 repair): A requirement's Release Classification tracks *when that implementation is scheduled* per `ROADMAP.md`'s phases, not merely whether a DS-001/ADR principle constrains it. Several requirements in this document are Committed/MVP because they are unconditional governance boundaries (e.g., DS-ARC-001's client/server separation, DS-ARC-011's TradeValidationPipeline boundary is enforced via the Committed DS-EXE-001 in DS-002 even though DS-ARC-011's own *implementation* is Planned/Phase 7). Where a requirement describes later-phase feature implementation (Phase 6 AI provider work, Phase 7 execution/broker work, Phase 3/4/5/9 features), it is classified Planned to match, even though the constraints governing it may already be Committed elsewhere.

3. Audience

Engineering contributors, independent auditors, and future Codex authors implementing or extending DarkSage's technical architecture.

4. Definitions

See DS-001 §24, DS-002 §4. Additional terms:

Term	Meaning
Architectural contract	A structural boundary or interface obligation (e.g., "clients never contain critical trading logic") that constrains implementation regardless of which specific technology fulfills it
Governing Source	The repository-root document/section (`ARCHITECTURE.md`, `PROJECT_SPEC.md`, `ROADMAP.md`) that already establishes a given architectural decision, cited per DS-002's Committed/MVP eligibility test
TradeValidationPipeline	The canonical 12-stage trade-proposal validation sequence defined once in `ARCHITECTURE.md` §14 and mirrored verbatim in `docs/pipeline-stages.txt`, checked by `scripts/verify-foundation.sh`

5. Client/Server Topology

DS-ARC-001 — Client/Server Separation

Priority: Critical | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Enforce that trading logic cannot leak into a client, protecting user authority and auditability regardless of which client surface is used.

Description: DarkSage shall use a client/server architecture in which desktop and mobile clients communicate with core backend services exclusively through a defined API; clients shall never contain critical trading logic, and the backend shall be the authoritative source of truth for trading and account state (positions, orders, portfolios, signals, strategies, risk state, broker state).

Dependencies: ADR-001 (Desktop-First); `ARCHITECTURE.md` §2, §30; `AGENTS.md` "Mobile/Backend Rule," "Desktop/Backend Rule"

Acceptance Criteria:

- Neither the desktop nor mobile client independently computes a trade decision, submits a broker order, or stores authoritative trading state.
- A client reads/displays backend-computed state; it does not duplicate the computation.
- Desktop and mobile, where both exist, observe the same backend-authoritative Auto-Trader/account state.

Edge Cases:

- Offline/degraded client operation (DS-OPS-004) reads last-known cached backend state; it does not fall back to independently computed trading logic.

Implementation Notes: Exact API contract is a DS-006 concern.

Testing: Architectural boundary test (static analysis / code-ownership check) confirming no trading-logic module exists under a client application directory.

DS-ARC-002 — Desktop Client

Priority: High | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Formalize the Phase 1 desktop shell as the primary current product surface, per ADR-001.

Description: DarkSage shall provide a desktop client (location: `apps/desktop/`) built on Electron, React, and TypeScript, serving as a client of the backend API, covering at minimum the Phase 1 scope: navigation, dashboard, scanner page, signal list, and signal detail.

Dependencies: ADR-001; `ARCHITECTURE.md` §3, §31; `ROADMAP.md` Phase 1; DS-WKS-001

Acceptance Criteria:

- The desktop client does not directly submit broker orders (DS-ARC-001).
- Phase 1 desktop surfaces (navigation, dashboard, scanner page, signal list/detail) are present and functional against the backend API.

Edge Cases: None beyond DS-ARC-001's general client boundary.

Implementation Notes: Full desktop responsibility list (Strategy Lab, Backtesting UI, Portfolio UI, Auto-Trader controls, Settings, Logs, Research tools, Sage AI interface) is `ARCHITECTURE.md` §3's authoritative reference; later phases add these surfaces incrementally as their backing features are built.

Testing: Desktop-launch smoke test (ROADMAP.md Phase 1 exit criterion: "Desktop app launches").

DS-ARC-003 — Mobile Client

Priority: Medium | **Release Classification:** Planned | **Status:** Draft

Purpose: Record the mobile client as approved future direction (Phase 9) without committing it to current scope.

Description: DarkSage should provide a mobile client (location: `apps/mobile/`, React Native, iPhone-first) as a client of the same backend API, per `ROADMAP.md` Phase 9, without running the full scanner, backtester, or execution engine locally.

Dependencies: ADR-001 (desktop-first does not prohibit future companion clients); `ARCHITECTURE.md` §4, §29; `ROADMAP.md` Phase 9; DS-MOB-001/002 (DS-002)

Acceptance Criteria (fixed now in the DS-004-A03 repair, per already-approved `ARCHITECTURE.md`/`AGENTS.md` boundaries — implementation timing remains Phase 9):

- The mobile client, whenever built, contains no local implementation of the scanner, backtester, or execution engine — it calls the backend API for these, per DS-ARC-001.
- The mobile client does not store authoritative trading/Auto-Trader state locally; it displays backend-authoritative state and can trigger backend state changes (e.g., stop Auto-Trader), never compute or hold the canonical state itself.
- Mobile and desktop, observing the same backend, show identical Auto-Trader/account state at the same point in time.

Edge Cases:

- Mobile offline/disconnected shows a last-known cached read-only snapshot, clearly marked as such, rather than a stale value indistinguishable from current (DS-PRD-008 analog).

Implementation Notes: `ARCHITECTURE.md` §4/§29 already specify responsibilities (dashboard, signals, portfolio monitoring, charts, watchlists, notifications, Sage interaction, Auto-Trader status, emergency stop, trade approvals) and the constraint that mobile must not independently run the Auto-Trader or store authoritative state (`AGENTS.md` "Mobile/Backend Rule"). Full implementation is Phase 9; the boundary above is testable in design/code-review terms even before Phase 9 begins.

Testing: Architectural boundary test (static analysis) once mobile code exists, confirming no trading-logic module under `apps/mobile/`; cross-client state-consistency test (Phase 9).

6. Backend Services

DS-ARC-004 — Backend Service Architecture

Priority: Critical | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Establish the backend as a modular, Python/FastAPI service that both current and future clients depend on identically.

Description: DarkSage shall implement backend services (location: `backend/`) using Python and FastAPI. `ARCHITECTURE.md` §5 defines the complete module tree (`api/`, `models/`, `market_data/`, `scanner/`, `indicators/`, `patterns/`, `strategies/`, `backtesting/`, `portfolio/`, `risk/`, `permissions/`, `execution/`, `monitoring/`, `notifications/`, `database/`, `audit/`, `security/`, `backend/app/knowledge/`) as the target end-state; this requirement's Committed/MVP scope covers only the Phase-1 subset below. Later modules may exist as reserved/stub directories but are not required to be functionally complete until their own phase.

Governing Source (narrowed in the DS-004-A01 repair): `ROADMAP.md` Phase 1 backend list.

Dependencies: `ARCHITECTURE.md` §5; `ROADMAP.md` Phase 1; DS-ARC-001

Acceptance Criteria:

- A FastAPI application skeleton exists and starts locally (Phase 1 exit criterion).
- The Phase-1 module subset — `api/`, `models/`, `market_data/`, `scanner/`, `indicators/`, `database/`, and `risk/` at foundation level only (deterministic calculations per DS-RSK-002, not yet wired into an execution pipeline) — is fully implemented and matches `ARCHITECTURE.md` §5's definition for those modules; no duplicate implementation of the same trading logic across modules (`AGENTS.md` "No Duplicate Business Logic").
- Modules outside the Phase-1 subset (`patterns/`, `strategies/`, `backtesting/`, `portfolio/`, `permissions/`, `execution/`, `monitoring/`, `notifications/`, `audit/`, `security/` beyond credential handling, `backend/app/knowledge/`) are not required to be functionally complete for this requirement to be satisfied; their completion is governed by their own phase-appropriate requirement (e.g., DS-ARC-010 for `backtesting/`, DS-ARC-012 for full `risk/`+`permissions/` pipeline integration).

Edge Cases:

- A module requiring functionality from another (e.g., `scanner/` using `indicators/`) does so via the module's public interface, not by reimplementing the calculation.

Implementation Notes: Full module responsibilities are `ARCHITECTURE.md` §5's authoritative reference.

Testing: Backend-startup smoke test (ROADMAP.md Phase 1 exit criterion: "Backend starts locally").

DS-ARC-005 — Shared Models

Priority: High | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Ensure every client and backend module operates on the same data shapes, preventing drift between desktop, mobile, and backend representations.

Description: DarkSage shall define shared data models (location: `shared/`). `ARCHITECTURE.md` §6 lists the complete target set (Candle, Quote, Signal, StrategyProfile, TradeDecision, Position, Portfolio, MarketRegime, RiskState, Order, BrokerState, ChartAnnotation, BacktestResult); this requirement's Committed/MVP scope covers only the five models `ROADMAP.md` Phase 1 explicitly names: Candle, Quote, Signal, StrategyProfile, TradeDecision. The remaining eight models (Position, Portfolio, MarketRegime, RiskState, Order, BrokerState, ChartAnnotation, BacktestResult) are Planned, matching their own DS-005 entity classifications (DS-DB-003, 006, 008–013).

Governing Source (narrowed in the DS-004-A01 repair): `ROADMAP.md` Phase 1 ("Core models," "Candle model," "Quote model," "Signal model," "StrategyProfile model," "TradeDecision model").

Dependencies: `ARCHITECTURE.md` §6; `ROADMAP.md` Phase 1; DS-DAT-003

Acceptance Criteria:

- Client and backend code reference the shared model definitions rather than independently redefining the same shape, for at minimum the five Phase-1 models.
- A shared-model change updates all affected clients/services/tests in the same change (`AGENTS.md` "Shared Contracts").
- The eight Planned models are not required to exist for this requirement to be satisfied.

Edge Cases: None beyond general schema-consistency concerns, delegated to DS-005.

Implementation Notes: DS-005 concern for the full schema/serialization detail.

Testing: Cross-client/backend schema-consistency test.

7. Market Data Architecture

DS-ARC-006 — Market Data Provider Abstraction

Priority: Critical | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Prevent vendor lock-in for market data, operationalizing DS-INT-001's provider-boundary principle.

Description: DarkSage shall access market data exclusively through provider adapters implementing a common interface (e.g., `get_quote()`, `get_candles()`, `get_historical_candles()`, `get_company_info()`, `get_fundamentals()`, `get_news()`, `get_market_status()`), flowing Provider → Adapter → Normalizer → Cache/Database → consuming feature (Scanner/Charts/Backtester/Strategies).

Dependencies: `ARCHITECTURE.md` §7; DS-MKT-001; DS-INT-001

Acceptance Criteria:

- No consuming feature module calls a vendor-specific API directly; all access is through the adapter interface.
- A second provider adapter can be added without changing consuming-feature code.

Edge Cases:

- A field available from one provider but not another is surfaced as optional/extension data in the normalized model, not a required field (consistent with DS-MKT-001's edge-case handling).

Implementation Notes: `ARCHITECTURE.md` §7 is the authoritative interface reference.

Testing: Provider-substitution test (fixture provider swap; confirm consuming features unaffected).

8. Indicator Engine

DS-ARC-007 — Single Shared Indicator Engine

Priority: Critical | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Guarantee that a chart, a scanner result, a backtest, and a strategy all agree on what an indicator's value is — a single-source-of-truth requirement, not a feature choice.

Description: DarkSage shall implement each technical indicator exactly once, in a shared indicator engine used identically by Charts, Scanner, Backtesting, Strategies, and (in later phases) the Paper and Live Auto-Trader. No feature may reimplement indicator math independently.

Dependencies: `ARCHITECTURE.md` §8; `ROADMAP.md` Phase 1 (SMA, EMA, RSI, MACD, ATR, Bollinger Bands, VWAP, ADX, OBV, Relative Volume, Relative Strength); DS-CHT-002; DS-PRD-004

Acceptance Criteria:

- Given identical input data, an indicator produces identical output whether invoked from the chart, scanner, or backtester.
- Each Phase 1 indicator has unit tests against known reference data (ROADMAP.md Phase 1 exit criterion: "Indicators match reference tests").

Edge Cases:

- Insufficient historical data for an indicator's lookback period is disclosed as such (consistent with DS-CHT-002's edge case), not silently computed as a misleading partial value.

Implementation Notes: Full indicator catalog (current and future) is `ARCHITECTURE.md` §8's authoritative reference.

Testing: Cross-consumer determinism regression test (chart vs. scanner vs. backtest, same input, same output); reference-data unit test suite.

9. Chart Architecture

DS-ARC-008 — Chart Adapter Abstraction

Priority: High | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Let users choose a chart renderer without the choice affecting analytical correctness — chart engines render, they do not calculate.

Description: DarkSage shall support Apache ECharts and TradingView Lightweight Charts behind a common chart-adapter abstraction; chart engines shall render data only and shall not independently calculate indicators, patterns, or strategy logic — DarkSage owns all chart data and calculations.

Dependencies: `ARCHITECTURE.md` §9; `ROADMAP.md` Phase 1; DS-CHT-001; DS-ARC-007

Acceptance Criteria:

- Both chart engines render identical market data, indicator values, and trade markers for the same input (ROADMAP.md Phase 1 exit criterion: "Both chart engines render the same data").
- Users can select the active chart engine; switching engines does not change any computed value shown.

Edge Cases:

- A visual feature supported by one renderer but not the other is disclosed as an engine-specific limitation, not silently omitted without indication.

Implementation Notes: `ARCHITECTURE.md` §9/§14 (PROJECT_SPEC) is the authoritative reference for supported overlays.

Testing: Cross-renderer data-parity regression test.

10. Strategy and Backtesting Architecture

DS-ARC-009 — Strategy Interface and Versioning

Priority: Medium | **Release Classification:** Planned | **Status:** Draft

Purpose: Give every strategy, regardless of family, a common, versioned, auditable shape — matching DS-STR's product-level requirements with the Phase 2 architectural contract.

Description: DarkSage should implement strategies against a common interface, each `StrategyProfile` carrying unique ID, name, version, status, supported timeframes/instruments, configuration, risk assumptions, and historical statistics, with defined statuses (Experimental, Watch, Active, Reduced, Suspended).

Dependencies: `ARCHITECTURE.md` §10; `ROADMAP.md` Phase 2; DS-STR-001

Acceptance Criteria:

- A strategy logic change increments its version and preserves prior results (`AGENTS.md` "No Silent Strategy Changes").
- Strategy status transitions are recorded, not silently applied.

Edge Cases: None beyond DS-STR-001's existing edge cases.

Implementation Notes: `ARCHITECTURE.md` §10/§11 is the authoritative reference for the full performance-tracking dimension list.

Testing: Version-increment/result-preservation regression test.

DS-ARC-010 — Backtest Engine Architecture

Priority: Medium | **Release Classification:** Planned | **Status:** Draft

Purpose: Ground DS-BKT's product-level backtesting requirements in a concrete, reusable engine architecture.

Description: DarkSage should implement backtesting via a historical data loader, event engine, simulated broker, fill/spread/slippage/fee models, portfolio accounting, the shared Risk Engine, and performance analytics — reusing production strategy logic (DS-ARC-009, DS-ARC-007) rather than a parallel implementation, and protecting against look-ahead bias, survivorship bias, future-known fundamentals, invalid fills, unrealistic liquidity assumptions, and data leakage.

Dependencies: `ARCHITECTURE.md` §13; `ROADMAP.md` Phase 2; DS-BKT-001, DS-BKT-002, DS-BKT-003

Acceptance Criteria:

- Backtests reuse the same strategy/indicator code paths as live analysis (DS-ARC-007, DS-ARC-009), not a duplicate implementation.
- Backtest results include realistic costs by default (ROADMAP.md Phase 2 exit criterion), consistent with DS-BKT-002.
- No look-ahead leakage is detectable in validation tests (ROADMAP.md Phase 2 exit criterion), consistent with DS-BKT-003.

Edge Cases: See DS-BKT-001/002/003's existing edge cases; this requirement adds no new ones at the architecture level.

Implementation Notes: `ARCHITECTURE.md` §13 is the authoritative component list.

Testing: Shared-logic reuse audit (backtest vs. live code path); look-ahead-bias regression fixture.

11. Canonical Trade Validation Pipeline

DS-ARC-011 — TradeValidationPipeline Enforcement

Priority: Critical | **Release Classification:** Planned (Phase 7, `ROADMAP.md`) | **Status:** Draft

Purpose: Formalize the repository's single most safety-critical architectural contract as a Codex requirement: the concrete mechanism through which ADR-002, DS-PRD-006, and DS-PRD-007 are actually enforced for any future executable trade proposal. Classified Planned in the DS-004-A01 repair because the full pipeline *implementation* is Phase 7 work; the unconditional governance boundary itself (no execution path may ever bypass it) is separately fixed as Committed in DS-EXE-001 (DS-002) so it cannot be silently weakened before Phase 7 arrives.

Description: Any executable trade proposal shall pass through the canonical `TradeValidationPipeline` in full — AI/Strategy Engine → Trade Proposal → Signal Validator → Strategy Validation → Risk Engine → Permissions Engine → Portfolio/Exposure Checks → Buying Power Checks → Market Condition Checks → Order Validation → Execution Engine → Broker Adapter — defined once in `ARCHITECTURE.md` §14 and mirrored exactly in `docs/pipeline-stages.txt`. No stage may be skipped, reordered, renamed, or duplicated. The AI/Strategy Engine has no authority to directly access or call the Execution Engine or Broker Adapter under any circumstance, regardless of confidence, urgency, or which AI provider generated the proposal.

Dependencies: ADR-002; ADR-003; DS-PRD-006; DS-PRD-007; DS-SGE-005, DS-SGE-006; `ARCHITECTURE.md` §14; `TRADING_RULES.md` Core Rules #2–#3; `PROJECT_SPEC.md` §2.3

Acceptance Criteria:

- `scripts/verify-foundation.sh`'s canonical-pipeline check continues to pass for every document that reproduces the pipeline (already enforced repository-wide; this requirement records the product/architecture obligation the script mechanically verifies).
- No executable trade proposal is enabled until the full pipeline exists, is implemented, and is independently reviewed (`ROADMAP.md` Phase 7 exit criterion).
- Duplicate-order and idempotency tests pass before any executable proposal path is enabled.

Edge Cases:

- Prior to Phase 7 (no executable proposals yet), the AI/Strategy Engine may still produce and explain a Trade Proposal for research/display purposes; it remains advisory-only and does not reach Order Validation, Execution Engine, or Broker Adapter.

Implementation Notes: `ARCHITECTURE.md` §14's stage table is the single authoritative definition; this requirement exists to give it Codex traceability, not to redefine it. Changes to the pipeline stages must update `ARCHITECTURE.md` §14, `docs/pipeline-stages.txt`, `PROJECT_SPEC.md`, `TRADING_RULES.md`, `SECURITY_RULES.md`, and `ROADMAP.md` together, consistent with the existing canonical-source convention.

Testing: `scripts/verify-foundation.sh` (existing, automated); adversarial bypass-attempt test (see DS-PRD-006); duplicate-order idempotency test (Phase 7).

12. Risk and Permissions Architecture

DS-ARC-012 — Risk Engine and Permissions Engine as Distinct Pipeline Stages

Priority: Critical | **Release Classification:** Planned (Phase 7, `ROADMAP.md`) | **Status:** Draft

Purpose: Ground DS-RSK-001's product-level Risk Engine authority requirement in the concrete pipeline position `ARCHITECTURE.md` establishes.

Description: The Risk Engine and Permissions Engine shall exist as distinct stages of the `TradeValidationPipeline` (DS-ARC-011): the Risk Engine evaluates trade- and account-level risk limits (max risk per trade, daily/weekly loss limits, strategy drawdown, volatility/liquidity/spread checks); the Permissions Engine evaluates instrument- and account-level permissions (allowed instrument category, signal-grade restrictions, account trading permissions). Neither may be bypassed, merged, or reordered relative to the pipeline.

Dependencies: DS-ARC-011; DS-RSK-001; DS-RSK-002; `ARCHITECTURE.md` §17, §18; `TRADING_RULES.md` Risk Rules

Acceptance Criteria:

- Risk and Permissions checks execute as separate, independently testable pipeline stages.
- A permissions failure (e.g., disallowed instrument category) is distinguishable in logs/audit trail from a risk failure (e.g., exceeded daily loss limit).

Edge Cases:

- Initial instrument-category scope is Stocks and ETFs where supported (`ARCHITECTURE.md` §18); Options, Crypto, and Futures are later-phase categories requiring their own Permissions Engine configuration before being allowed.

Implementation Notes: ARCHITECTURE.md §17/§18 and TRADING_RULES.md are the authoritative rule-content references.

Testing: Stage-isolation test confirming Risk and Permissions failures are independently attributable.

13. AI Architecture

DS-ARC-013 — AI Provider Interface

Priority: Critical | **Release Classification:** Planned (Phase 6, ROADMAP.md) | **Status:** Draft

Purpose: Give DS-PRD-001/DS-AI-005/DS-SGE-018's model-independence mandate a concrete implementation contract.

Description: All AI providers, local and cloud, shall implement a common interface (e.g., `complete()`, `chat()`, `stream()`) so that feature code (Sage, signal analysis, research summaries, strategy explanation) and the AI orchestrator never depend on a vendor-specific SDK directly. Adding, removing, or replacing a provider shall require only a new adapter implementing the existing interface, not changes to feature code.

Dependencies: DS-PRD-001; DS-AI-005; DS-SGE-018; ARCHITECTURE.md §22; ROADMAP.md Phase 6

Acceptance Criteria:

- Feature code contains no vendor-specific (e.g., OpenAI-SDK-specific) types or calls; all AI interaction routes through the common interface.
- Per-feature provider/model selection (Sage chat, deep signal analysis, research/news summaries, strategy explanations) is configured in the backend, not the client, consistent with DS-ARC-001.

Edge Cases:

- A provider lacking a capability the interface exposes (e.g., no streaming) degrades per DS-AI-006/DS-SGE-019, not a silent capability change.

Implementation Notes: ARCHITECTURE.md §22/§24 is the authoritative reference for initial adapters (local/llama.cpp-compatible, OpenAI, Anthropic, Google Gemini, custom OpenAI-compatible).

Testing: Cross-provider parity regression suite (DS-SGE-018).

DS-ARC-014 — Local-First AI

Priority: High | **Release Classification:** Planned | **Status:** Draft

Purpose: Formalize local AI as the default per PROJECT_SPEC.md §2.1's Cheap-First principle, without committing the full Phase 6 feature set to current scope.

Description: DarkSage should default to local AI models for routine advisory tasks; cloud AI shall remain optional, user-configured with the user's own API key, and shall never be required for basic application operation or for any deterministic calculation (scanning, indicators, risk, backtesting, portfolio math, trade validation).

Dependencies: PROJECT_SPEC.md §2.1, §23; ARCHITECTURE.md §23, §24; DS-PRD-004; ROADMAP.md Phase 6

Acceptance Criteria:

- The application functions fully with zero cloud AI providers configured (`ROADMAP.md` Phase 6 exit criterion).
- No deterministic feature's correctness depends on any AI provider being configured or available.

Edge Cases: See DS-AI-006/DS-SGE-019 for degradation behavior when local AI itself is unavailable.

Implementation Notes: `ARCHITECTURE.md` §23 is the authoritative runtime reference (llama.cpp-compatible/ONNX/other efficient local runtime).

Testing: Zero-cloud-provider functional test.

DS-ARC-015 — AI Provider Credential Handling

Priority: Critical | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Apply DS-SEC-001's credential-handling mandate to the specific, named case of AI provider API keys, per `SECURITY_RULES.md`.

Description: AI provider API keys shall never be committed to source control, logged (including debug/verbose logs), or exposed in frontend/client source or bundled JavaScript; production credentials shall use OS credential storage or an encrypted secrets vault; development may use `.env` files excluded by `.gitignore`; no key shall ever be sent to a provider other than the one the user configured it for.

Dependencies: DS-SEC-001; `SECURITY_RULES.md` "AI Privacy and Provider Credentials"; `ARCHITECTURE.md` §24

Acceptance Criteria:

- No provider API key appears in commits, logs, or frontend/client source (`ROADMAP.md` Phase 6 exit criterion).
- A stored key is never redisplayed in full once saved, in any future Settings > AI Providers UI.

Edge Cases: None beyond DS-SEC-001's existing edge cases.

Implementation Notes: `SECURITY_RULES.md` is the authoritative security-control reference; `docs/secret-scanning.md` documents the repository's automated scanning.

Testing: Secret-scanning test (existing repository tooling); credential-exposure audit across frontend build output.

14. Database and Caching

DS-ARC-016 — Database Strategy

Priority: High | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Avoid premature infrastructure complexity, per `ARCHITECTURE.md` §20's explicit "do not introduce infrastructure complexity until needed" principle.

Description: DarkSage shall use SQLite as the initial database. Migration to PostgreSQL (and TimescaleDB, only if justified) is a later-phase decision (`ROADMAP.md` Phase 12), not committed to current scope.

Dependencies: ARCHITECTURE.md §20; ROADMAP.md Phase 1, Phase 12; DS-DAT-001

Acceptance Criteria:

- Phase 1 backend operates against SQLite without requiring an external database service.
- No Committed/MVP requirement depends on PostgreSQL-specific or TimescaleDB-specific behavior.

Edge Cases: A future migration (Phase 12) requires a reviewed migration/backup path (AGENTS.md "Database Changes"), not an in-place undocumented switch.

Implementation Notes: DS-005 concern for schema detail.

Testing: Local SQLite functional test (Phase 1).

DS-ARC-017 — Caching Strategy

Priority: Medium | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Apply the same anti-premature-complexity principle to caching.

Description: DarkSage shall use in-memory caching and local persistence initially; a dedicated caching service (e.g., Redis) shall not be added until a demonstrated need exists.

Dependencies: ARCHITECTURE.md §21; AGENTS.md "External Services"

Acceptance Criteria:

- No Committed/MVP requirement depends on a dedicated caching service being deployed.

Edge Cases: None recorded.

Implementation Notes: DS-005 concern for cache-key/invalidation detail if/when introduced.

Testing: Not separately tested beyond the features that rely on caching behavior.

15. Deployment Stages

DS-ARC-018 — Staged Deployment Model

Priority: Medium | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Keep recurring cost near zero during development, per PROJECT_SPEC.md §2.1's Cheap-First Architecture principle.

Description: DarkSage shall follow a staged deployment model: Stage 1 (local development, target hosting cost $0/month) → Stage 2 (paper testing, backend may remain local) → Stage 3 (hosted backend for critical services) → Stage 4 (live trading, requiring hardened security, broker reconciliation, monitoring, fail-safe controls, kill switches, deployment review, and audit logging). Stage 4 is not committed to current scope.

Dependencies: ARCHITECTURE.md §27, §30; ROADMAP.md Phase 1, Phase 12–13; PROJECT_SPEC.md §2.1

Acceptance Criteria:

- Stage 1 development requires no paid hosting service.
- Progression to a later stage requires its documented prerequisites (e.g., Stage 4's security/reconciliation/kill-switch review) to be satisfied first, not skipped for convenience.

Edge Cases: None beyond the staged prerequisites themselves.

Implementation Notes: ARCHITECTURE.md §27/§30 and ROADMAP.md Phase 12–14 are the authoritative staging references.

Testing: Stage-1 zero-cost local-run verification.

16. Core Architectural Rules

DS-ARC-019 — No Tight Coupling

Priority: High | **Release Classification:** Committed / MVP | **Status:** Draft

Purpose: Formalize ARCHITECTURE.md §30's explicit anti-coupling list as a testable architectural requirement, since it is the structural precondition for DS-ARC-006/007/008/013's abstraction boundaries actually holding.

Description: DarkSage shall not tightly couple: UI and trading logic; strategy logic and broker APIs; chart renderers and indicator calculations; market-data providers and core models; AI providers and deterministic systems; or desktop state and Auto-Trader state. Every major external dependency shall be replaceable through an interface or adapter.

Dependencies: ARCHITECTURE.md §30; DS-ARC-001, DS-ARC-006, DS-ARC-007, DS-ARC-008, DS-ARC-013

Acceptance Criteria:

- Each listed coupling pair is verifiable as absent via module-dependency inspection (e.g., no direct import from a chart-rendering module into indicator-calculation code).

Edge Cases: None beyond the specific pairs listed.

Implementation Notes: This requirement is the structural precondition underlying several other DS-ARC requirements' testability; violations here would undermine DS-ARC-006/007/008/013 even if those pass their own individual tests.

Testing: Module-dependency/import-boundary static analysis.

16a. Missing Architecture Contracts (added in the DS-004-A02 repair)

The following close a completeness gap identified by independent audit: ARCHITECTURE.md already specifies these contracts with no prior DS-004 coverage.

DS-ARC-020 — Strategy Performance and Strategy DNA Architecture

Priority: Medium | **Release Classification:** Planned | **Status:** Draft

Governing Source: ARCHITECTURE.md §11, §12; ROADMAP.md Phase 3; DS-PERF-001/002/003 (DS-002)

Purpose: Give Strategy Performance Intelligence (DS-PERF) a concrete storage/computation architecture.

Description: DarkSage should compute and store strategy performance metrics and Strategy DNA from the same statistical-validation infrastructure used for backtesting (DS-ARC-010), segmented per DS-PERF-002's dimensions, using measured statistical evidence only — a model's guess is never a substitute for computed statistics.

Dependencies: DS-ARC-010; DS-PERF-001/002/003; DS-DB-003, DS-DB-006 (DS-005)

Acceptance Criteria: Deferred to Phase 3 authoring; the "statistical evidence, not AI opinion" constraint is fixed now.

Edge Cases: None recorded at this classification level.

Implementation Notes: `ARCHITECTURE.md` §11/§12 is authoritative.

Testing: Not yet applicable — Phase 3.

DS-ARC-021 — Broker and Auto-Trader Architecture

Priority: Critical | **Release Classification:** Planned | **Status:** Draft

Governing Source: `ARCHITECTURE.md` §14, §15; `ROADMAP.md` Phase 7; DS-EXE-003/006 (DS-002)

Purpose: Give the Auto-Trader state machine and Broker Adapter a concrete architecture, distinct from (and downstream of) the TradeValidationPipeline's validation stages (DS-ARC-011).

Description: DarkSage should implement Auto-Trader state (disabled/enabled/paused/emergency_stop) as backend-authoritative state observed identically by all clients (DS-ARC-001), and should implement broker connectivity through the common Broker Adapter interface (DS-EXE-006) with PaperBroker as the initial implementation.

Dependencies: DS-ARC-001; DS-ARC-011; DS-EXE-003, DS-EXE-006

Acceptance Criteria: Deferred to Phase 7 authoring; the backend-authoritative-state and adapter-interface obligations are fixed now per DS-ARC-001's already-Committed boundary.

Edge Cases: None beyond DS-ARC-001's existing edge cases.

Implementation Notes: `ARCHITECTURE.md` §14/§15 is authoritative.

Testing: Not yet applicable — Phase 7.

DS-ARC-022 — Emergency Stop and Emergency Flatten Architecture

Priority: Critical | **Release Classification:** Planned | **Status:** Draft

Governing Source: `ARCHITECTURE.md` §16; `TRADING_RULES.md`/`SECURITY_RULES.md` "Emergency Stop"/"Emergency Flatten"; `ROADMAP.md` Phase 7; DS-EXE-004/005 (DS-002)

Purpose: Give the two emergency controls a concrete, independently-reachable architecture so they cannot be blocked by the same failure that necessitates using them.

Description: DarkSage should implement Emergency Stop and Emergency Flatten as backend operations reachable from any authorized client independent of the normal order-submission code path (so a partial system failure that would justify triggering them does not also disable them), with Emergency Flatten requiring strong authentication in live mode.

Dependencies: DS-ARC-021; DS-EXE-004, DS-EXE-005

Acceptance Criteria: Deferred to Phase 7 authoring; the independent-reachability requirement is fixed now given its safety-critical nature.

Edge Cases: None recorded at this classification level.

Implementation Notes: `ARCHITECTURE.md` §16 is authoritative.

Testing: Not yet applicable — Phase 7 exit criteria ("Emergency Stop passes tests").

DS-ARC-023 — Portfolio Architecture

Priority: Medium | **Release Classification:** Planned | **Status:** Draft

Governing Source: `ARCHITECTURE.md` §19; `ROADMAP.md` Phase 4; DS-PRT-001..004 (DS-002, Planned)

Purpose: Give portfolio tracking (holdings, cash, exposure, sector allocation, correlations, risk budgets, performance, benchmarks, rebalancing, goal tracking) a concrete service architecture.

Description: DarkSage should implement portfolio services as backend-authoritative computations over the Transaction/Position data model (DS-005, once the Transaction entity is added per DS-005 Appendix A), reusing the deterministic calculation engine (DS-PRD-004) rather than a parallel implementation.

Dependencies: DS-DB-008/009 (DS-005); DS-PRT-001..004

Acceptance Criteria: Deferred to Phase 4 authoring.

Edge Cases: None recorded at this classification level.

Implementation Notes: `ARCHITECTURE.md` §19 is authoritative.

Testing: Not yet applicable — Phase 4.

DS-ARC-024 — Mobile Control Security Architecture

Priority: High | **Release Classification:** Planned | **Status:** Draft

Governing Source: `ARCHITECTURE.md` §26; `SECURITY_RULES.md` "Desktop and Mobile Security"; `ROADMAP.md` Phase 9; DS-MOB-003 (DS-002)

Purpose: Ensure mobile's ability to control Auto-Trading is architected with security proportionate to its risk, not bolted on afterward.

Description: DarkSage should require strong authentication for high-risk mobile actions (pause Auto-Trade, emergency stop, trade approvals) and should use secure platform storage (iOS Keychain) rather than storing broker secrets directly on the device.

Dependencies: DS-ARC-003; DS-ARC-015 (credential handling pattern); DS-MOB-003

Acceptance Criteria: Deferred to Phase 9 authoring; the secure-storage/strong-auth obligations are fixed now per `SECURITY_RULES.md`.

Edge Cases: None recorded at this classification level.

Implementation Notes: `SECURITY_RULES.md` is authoritative.

Testing: Not yet applicable — Phase 9.

DS-ARC-025 — Trading Knowledge Engine Architecture

Priority: Low | **Release Classification:** Planned | **Status:** Draft

Governing Source: ARCHITECTURE.md §31 (complete); PROJECT_SPEC.md §33; ROADMAP.md Phase 5; DS-EDU (DS-002)

Purpose: Formalize the complete Trading Knowledge Engine architecture (backend/app/knowledge/ — the single authoritative location, always referenced with the full path), which DS-004 v0.2.0 omitted entirely despite ARCHITECTURE.md §31 specifying it in detail.

Description: DarkSage should implement the Trading Knowledge Engine as deterministic local code (backend/app/knowledge/ingestion/, concepts/, patterns/, scoring/, provenance/, education/) that structures, scores, retrieves, explains, and statistically validates trading concepts — reusing existing pattern-detection (patterns/) and indicator (indicators/) modules rather than duplicating their math (AGENTS.md "No Duplicate Business Logic"). Detection, setup-quality scoring, and trade eligibility remain three distinct, non-conflatable steps; a high setup-quality score never grants, skips, or shortcuts any TradeValidationPipeline stage (DS-ARC-011). AI involvement is limited to natural-language explanation, semantic retrieval, tutoring, and summarization — never a substitute for the deterministic scoring/validation.

Dependencies: ARCHITECTURE.md §31 (complete); DS-EDU-001/002 (DS-002); DS-ARC-011; DS-PRD-004

Acceptance Criteria: Deferred to Phase 5 authoring; the detection/quality/eligibility separation and local-first/deterministic-first principles are fixed now per ARCHITECTURE.md §31's existing specification.

Edge Cases:

- Educational-source concepts (source, date, category, confidence, staleness, verification-required) are never presented as current authoritative fact without verification, and never exempted from the same statistical validation as any other strategy/pattern source (TRADING_RULES.md).

Implementation Notes: ARCHITECTURE.md §31 is the complete authoritative reference; this requirement exists to give it Codex traceability, not to redefine it.

Testing: Not yet applicable — Phase 5 exit criteria (ROADMAP.md Phase 5).

16b. Research Intelligence Architecture (added in the independent-audit H1 repair)

DS-ARC-026 — Research Intelligence Architecture

Priority: High | **Release Classification:** Planned | **Status:** Draft

Governing Source: DS-RSH-001 through DS-RSH-006 (DS-002, Planned); DS-001 §26.4

Purpose: Give Research Intelligence a concrete backend architecture — ingestion adapters per approved evidence domain, an evidence store preserving source/timestamp/provenance, and a thesis-monitoring service — closing the independent-audit H1 gap where DS-RSH existed only as DS-002 requirements with no architectural contract.

Description: DarkSage should implement Research Intelligence as provider-neutral ingestion adapters (news, filings, earnings, macro, insider/political disclosure, analyst revisions — DS-RSH-002) behind a common Research Source Adapter interface, mirroring DS-ARC-006's

market-data provider-abstraction pattern, normalizing each item into a ResearchEvidence record (DS-DB-029) before it reaches any consuming feature. A Thesis Monitoring service (backing DS-RSH-004) evaluates registered theses against newly ingested evidence and flags material changes without rewriting the stored original thesis. Sage's bounded research workflow (DS-RSH-006; DS-003 §6) invokes this architecture only through registered tools; it never ingests or fabricates evidence outside the adapter path.

Dependencies: DS-ARC-006 (adapter-pattern precedent); DS-RSH-001 through DS-RSH-006; DS-DB-029, DS-DB-030, DS-DB-031 (DS-005); DS-PRD-002

Acceptance Criteria:

- Each research domain (DS-RSH-002) is ingested through its own adapter; disabling one domain does not block or corrupt another.
- Every stored ResearchEvidence record retains its original source, publication/event time, and retrieval time, immutable once stored except through an explicit, logged correction.
- Thesis Monitoring evaluates new evidence against a stored thesis's assumptions/invalidation conditions and surfaces a flagged change without overwriting the thesis's original text (DS-RSH-004).
- A domain adapter's unavailability degrades to disclosed absence (DS-RSH-002's edge case), never silent gap-filling.

Edge Cases: A source returning conflicting evidence for the same claim is stored as multiple ResearchEvidence records, not collapsed into one (DS-RSH-005).

Implementation Notes: DS-005 (DS-DB-029/030/031) defines the storage schema; DS-006 (DS-API-RSH-001 through 003) defines the client-facing contract.

Testing: Adapter-substitution test per research domain (shared pattern with DS-ARC-006's own test); thesis-monitoring change-detection regression test; evidence-immutability audit.

16c. Journal & Review Architecture (added in the independent-audit H1 repair)

DS-ARC-027 — Journal & Review Architecture

Priority: High | **Release Classification:** Planned | **Status:** Draft

Governing Source: DS-JRN-001 through DS-JRN-005, DS-JRN-007 (DS-002, Planned); DS-JRN-006 (DS-002, Future/Exploratory — referenced for family context only, not an architectural commitment); DS-001 §26.3

Purpose: Give Journal & Review Intelligence a concrete backend architecture — an append-only journal-entry store preserving the original plan, a review-generation service, and a clear boundary between deterministic performance data and Sage's advisory commentary — closing the independent-audit H1 gap.

Description: DarkSage should implement the Trading Journal as an append-only JournalEntry store (DS-DB-032) where the original thesis/plan (DS-JRN-002) is written once and never overwritten; later annotations, Sage observations, and outcome records are appended as timestamped, attributed amendments referencing the original entry, never in-place edits. Daily and Weekly Review generation (DS-JRN-003/004, backed by DS-DB-033) reuses DS-PERF's and

DS-PRT's deterministic performance-calculation services for every quantitative figure it presents; Sage may draft narrative commentary around those figures but never substitutes its own text for a deterministic value (the DS-PRD-004 boundary applies identically here).

Dependencies: DS-ARC-009, DS-ARC-010 (deterministic-reuse pattern); DS-JRN-001 through DS-JRN-007; DS-DB-032, DS-DB-033 (DS-005); DS-PRD-004

Retention and deletion architecture (added for independent-audit Blocker 3): DS-JRN-007's user-controlled retention/deletion is implemented as a propagating delete over JournalEntry, its attachment references, and any DailyReview/WeeklyReview or Sage-drafted content that would reproduce the deleted entry's private text — never over the immutable Transaction/AuditLogEntry records a journal entry may reference, which remain a structurally distinct, independently-retained data class. Deletion is logged (DS-OPS-001) by identifier/timestamp only, never by reproducing the deleted content in the log entry itself.

Acceptance Criteria:

- A confirmed JournalEntry's original plan fields are immutable; every later change is a new, attributed, timestamped amendment record linked to the original (mirrors DS-DB-023's append-only pattern).
- Daily/Weekly Review quantitative figures trace to the same deterministic services DS-PERF/DS-PRT already define; no review figure is Sage-generated.
- Sage's review commentary is clearly separable from the deterministic figures it discusses (DS-JRN-005).

Edge Cases: A journal entry for a decision *not* to trade is stored with the same immutability/amendment discipline as an executed trade's entry (DS-JRN-001).

Implementation Notes: DS-005 (DS-DB-032/033) defines storage; DS-006 (DS-API-JRN-001 through 003) defines the API contract; DS-008 (DS-SCA-028) governs journal-data protection given its behavioral/psychological content sensitivity.

Testing: Append-only/amendment-chain integrity test (shared pattern with DS-DB-023/025's own tests); deterministic-figure-reuse audit confirming no review statistic is independently computed outside DS-PERF/DS-PRT.

16d. Canonical Trade Intelligence Package Service (added in the independent-audit H1 repair)

DS-ARC-028 — Canonical Trade Intelligence Package Service

Priority: Critical | **Release Classification:** Planned | **Status:** Draft

Governing Source: DS-SIG-005 (DS-002, Planned); DS-004 §11 (DS-ARC-011)

Purpose: Give the canonical Trade Intelligence Package (DS-SIG-005) a single backend service producing and versioning it, so Sage, charts, journal, validation, execution, alerts, and audit all reference the identical object rather than each recomputing or reconstructing an equivalent one independently — closing the independent-audit H1 gap where the package existed only as a

DS-002 requirement with no architectural owner.

Description: DarkSage should implement a single Trade Intelligence Package service that assembles a package's fields (DS-SIG-005's full field list) from their respective authoritative sources — Signal (DS-DB-004) for symbol/direction/confidence, the Risk Engine (DS-RSK-002) for capital-at-risk/risk-reward, StrategyProfile (DS-DB-005) for setup/strategy version, Research Intelligence (DS-ARC-026) for catalysts/evidence/contradictions, and the active automation mode (DS-EXE-008) for permission/automation state — never generating a deterministic field itself. Every package carries a stable identifier and version (DS-DB-028) referenced identically by every consuming surface (DS-CHT-006's chart overlays, DS-JRN's journal capture, DS-API-EXE-005's proposal creation, DS-ALT's alerts).

Dependencies: DS-ARC-011 (TradeValidationPipeline, the package's downstream consumer); DS-SIG-005; DS-DB-028; DS-PRD-002, DS-PRD-004

Acceptance Criteria:

- No consuming surface (chart, journal, validation, execution, alert, audit) recomputes a package field independently — each reads the same versioned package object.
- A field with no current value (e.g., no assigned strategy yet) is explicit/null in the package, never fabricated (DS-SIG-005's own acceptance criterion).
- Deterministic fields (capital at risk, risk/reward, position size) trace to their owning engine's calculation, never to generated text.

Edge Cases: A package referenced by an expired Signal (DS-SIG-004) retains its own historical field values; it is not silently regenerated against current market conditions.

Implementation Notes: DS-005 (DS-DB-028) defines the schema; DS-006 (DS-API-TIP-001) defines the read/reference contract; DS-007 (DS-UX-025) defines cross-surface presentation consistency; DS-008 (DS-SCA-029, added for independent-audit Blocker 2) defines the package's integrity, provenance, tamper-detection, authorization, and fail-closed security authority — this service never writes a deterministic field without an auditable call to the owning engine, per DS-SCA-029's no-unauthorized-override rule.

Testing: Cross-surface consistency test confirming chart, journal, and alert renderings of the same package ID show identical field values; field-provenance audit confirming every deterministic field traces to its owning engine.

17. Non-Goals

Consistent with `AGENTS.md` "Scope Control," DS-004 does not authorize, without explicit separate approval:

- replacing core frameworks (Electron/React/TypeScript, Python/FastAPI, SQLite) once adopted;
- changing broker architecture;
- changing database strategy (SQLite → PostgreSQL) ahead of its scheduled phase;
- removing any safety system described in this document;
- adding live trading capability;
- adding paid dependencies or unnecessary cloud infrastructure.

18. Dependencies

- DS-001 — Executive Vision & Product Foundation
- DS-002 — Software Requirements Specification, including the `DS-SIG`, `DS-EXE`, `DS-PERF`, `DS-MOB` families added in the DS-002-A03 repair
- DS-003 — Sage AI Bible
- ADR-001 through ADR-004
- `ARCHITECTURE.md`, `PROJECT_SPEC.md`, `ROADMAP.md`, `TRADING_RULES.md`, `SECURITY_RULES.md`, `AGENTS.md` (repository root — primary source for this document)

19. Risks and Constraints

- **Duplication risk:** the greatest risk to this document is drifting out of sync with `ARCHITECTURE.md` if the two are edited independently. Mitigation: DS-004 requirements cite `ARCHITECTURE.md` sections as Governing Source rather than re-deriving content, and any future architecture change should update both together.
- **Sequencing risk:** DS-004 is authored before DS-005/DS-006/DS-007/DS-008; several Implementation Notes delegate schema/API/UI/security detail forward, consistent with DS-002/DS-003's same pattern.
- **Classification discipline:** applied the same Committed/MVP eligibility test as DS-002/DS-003, now explicitly including `ARCHITECTURE.md`/`PROJECT_SPEC.md`/`ROADMAP.md` as authoritative controlled sources per the DS-002 v0.4.0 reconciliation.

20. Verification Approach

Each DS-ARC requirement states its own Testing. `scripts/verify-foundation.sh` already mechanically enforces the `TradeValidationPipeline`'s canonical-source consistency (DS-ARC-011) across `ARCHITECTURE.md`, `PROJECT_SPEC.md`, `TRADING_RULES.md`, `SECURITY_RULES.md`, and `ROADMAP.md`. Document-level verification (unique-ID check, cross-reference consistency, Release Classification consistency) is recorded in `.ai-workflow/HANDOFF.md` for this task.

21. References

- `docs/CODEX_INDEX.md`, `docs/standards/*`
- `ARCHITECTURE.md`, `PROJECT_SPEC.md`, `ROADMAP.md`, `TRADING_RULES.md`, `SECURITY_RULES.md`, `AGENTS.md`
- `docs/pipeline-stages.txt`, `scripts/verify-foundation.sh`
- `docs/codex/Volume-01-Foundation/DS-001-Executive-Vision.md`
- `docs/codex/Volume-02-Product/DS-002-SRS.md`
- `docs/codex/Volume-03-Sage/DS-003-Sage-AI-Bible.md`
- `docs/codex/Volume-12-ADRs/ADR-001-Desktop-First-Application.md` through `ADR-004-Presentation-Independence.md`

Appendix A — Open Questions

1. **PostgreSQL/TimescaleDB migration trigger** — `ARCHITECTURE.md` §20 says TimescaleDB only "if justified," without defining the trigger condition. Routine implementation

detail, appropriately deferred to whenever Phase 12 begins, not an SRS-level blocker.

2. **Live-trading hosting provider (Stage 3/4)** — not yet selected anywhere in the repository; genuinely unresolved but correctly out of current scope (Stage 4 is explicitly not committed).

3. **DS-004 vs. `ARCHITECTURE.md` change-control process** — now that both exist, a future process decision is needed on whether `ARCHITECTURE.md` remains independently editable or becomes fully subordinate to DS-004's Codex change-control (Draft → Under Review → Approved). Recorded here as a governance question for the owner; not resolved by this draft, and not blocking DS-004's own use in the interim (DS-004 cites `ARCHITECTURE.md`, so either document being current keeps them consistent).

4. **Phase-to-classification mapping precision** — the DS-004-A01 repair mapped requirements to Committed/Planned based on the best reading of `ROADMAP.md`'s phase boundaries and `AGENTS.md`'s document-priority order where `PROJECT_SPEC.md` and `ROADMAP.md` differ in granularity (e.g., `PROJECT_SPEC.md` §32's terse "Risk-engine foundation" Phase 1 mention vs. `ROADMAP.md`'s more detailed phase breakdown). This mapping is a defensible reading, not an owner-confirmed one; flagged for owner review alongside Open Question #3.

22. Founder Vision Completion Architecture

The architecture shall include bounded-agent orchestration, research ingestion/evidence services, thesis monitoring, canonical Trade Intelligence Package services, journal/review services, and notification-provider adapters. Agent orchestration may call only registered tools through policy enforcement and audit boundaries. Research and generative services cannot write authoritative deterministic financial fields.

Discord delivery shall use a notification adapter isolated from trading execution. Webhook/bot credentials are secret-managed; delivery is idempotent, rate-limit aware, retry bounded, and backed by an authoritative in-app event record. No Discord adapter method may invoke order approval, execution, modification, cancellation, Risk Engine configuration, or emergency-control release.